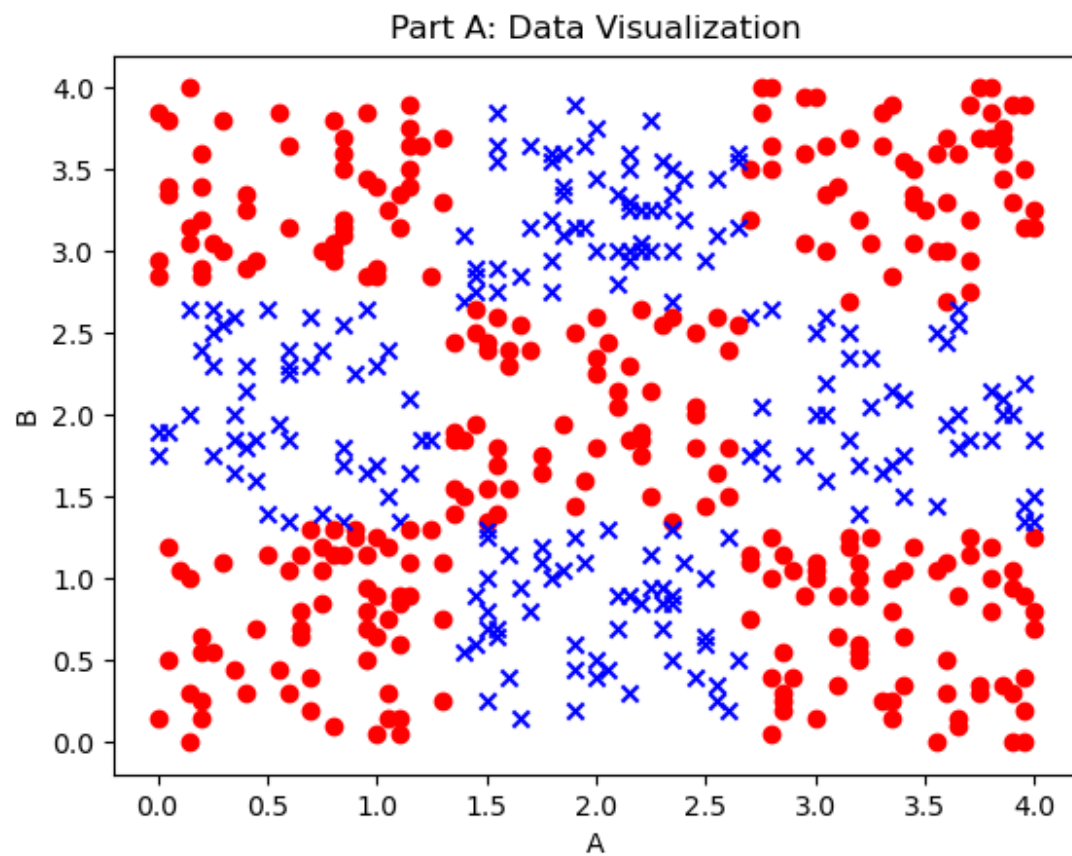


Part I: Classification

Part A: Scatter Plot of Dataset:

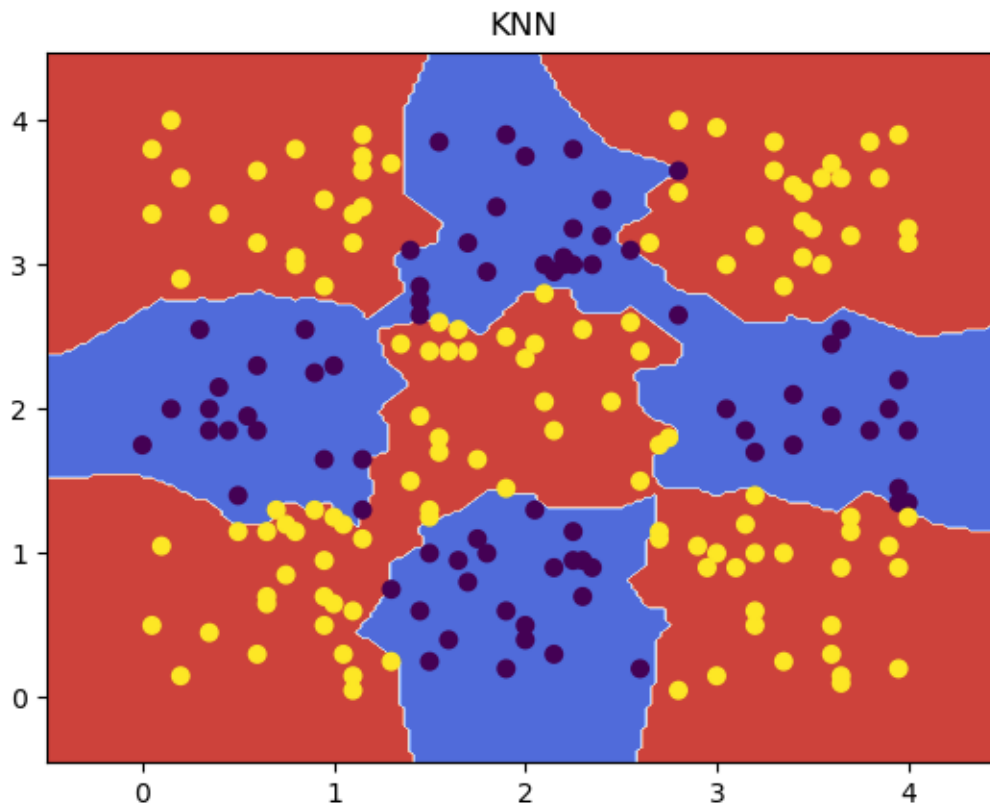


Part C: Building Classifiers

Name	Best Training Score	Testing Score
Nearest Neighbors	0.926667	0.945
Logistic Regression	0.573333	0.59
Decision Tree	0.98	0.99
Random Forest	0.903333	0.94
AdaBoost	0.67	0.6

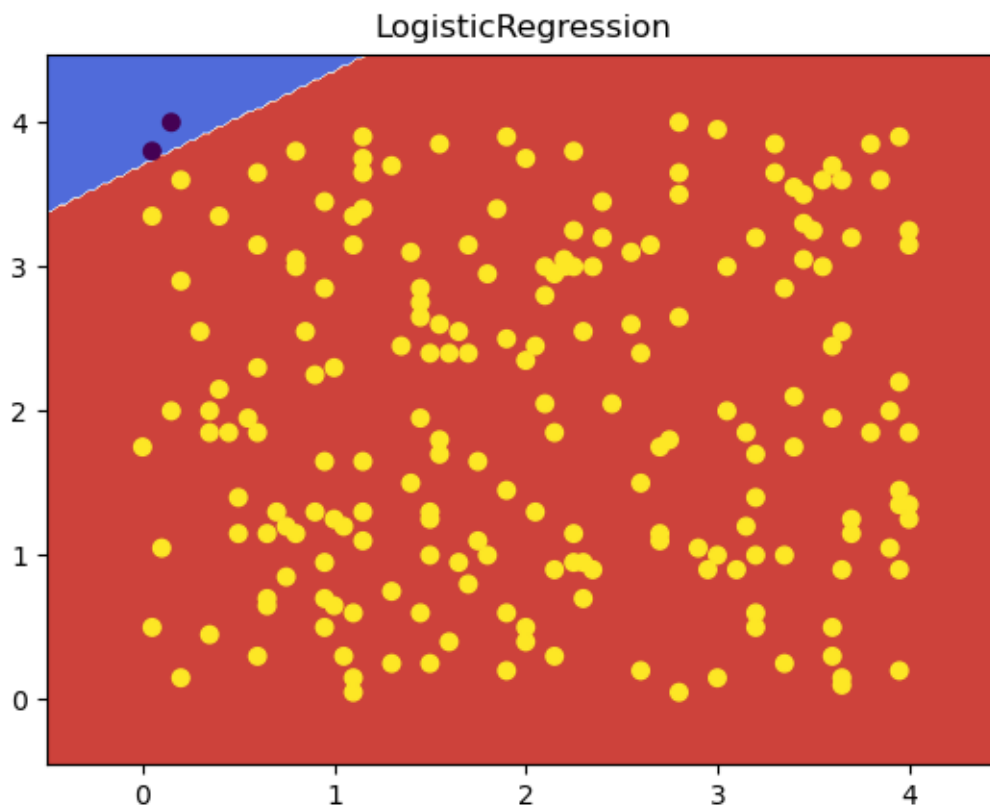
The decision tree classifier had the best results with a testing score of 99%.

1. K-Nearest Neighbors



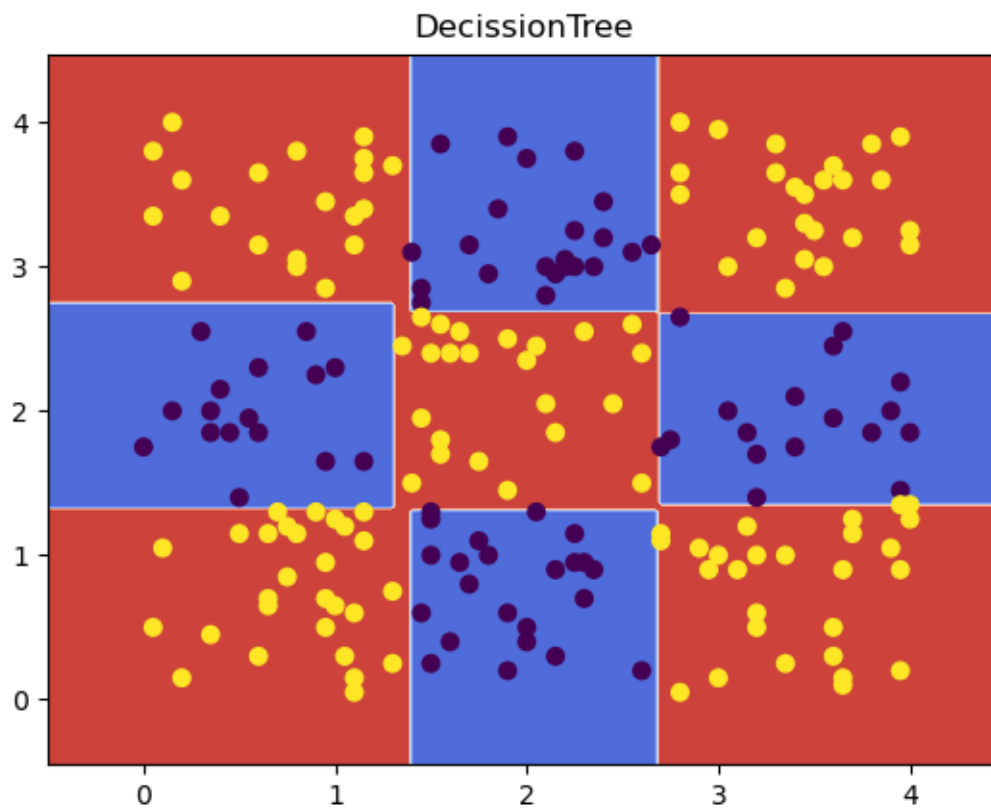
The K-Nearest Neighbors classifier worked very well as it was 95% accurate. The K-Nearest Neighbor classifier will assign the test node to the value of the 5 closest training nodes. This worked very well, especially as there seemed to be no erroneous data, or a large number of dimensions.

2. Logistic Regression



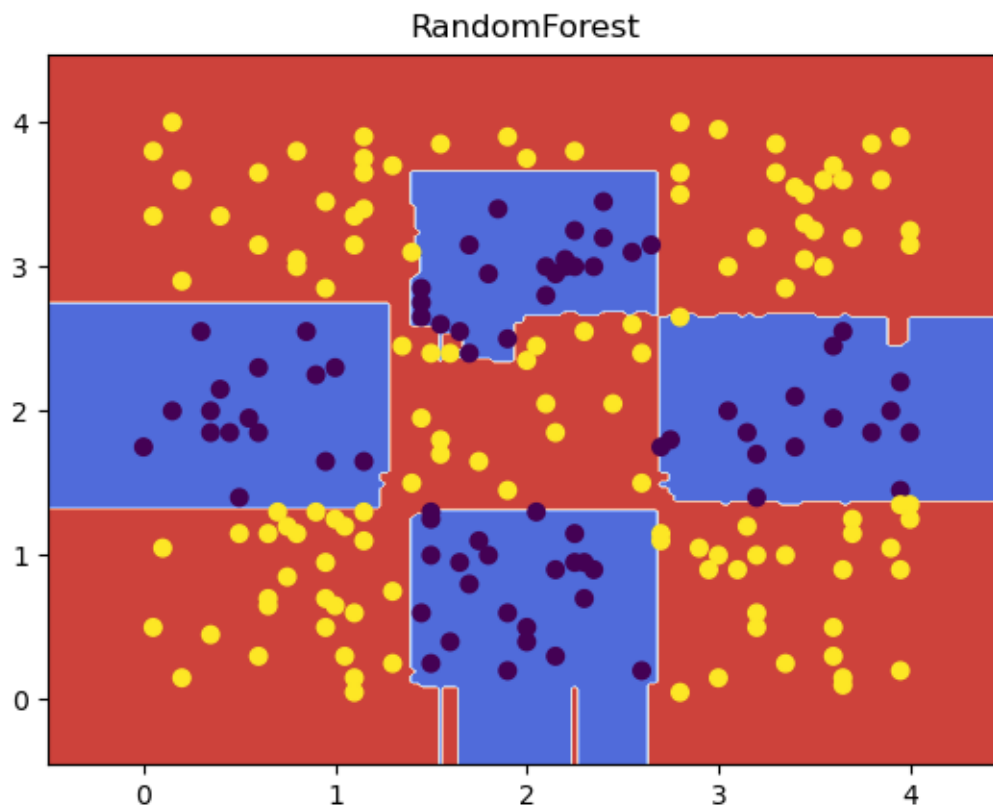
The logistic regression classifier did not work well at all for this data set, having an accuracy of 59%. The logistic regression classifier is a great classifier for binary tasks, as it is a linear classifier. This is perfect for spam filters where the options are binary. However, the tik-tac-toe pattern is not linearly separable, making it hard for the logistic regression algorithm to properly function.

3. Decision Trees



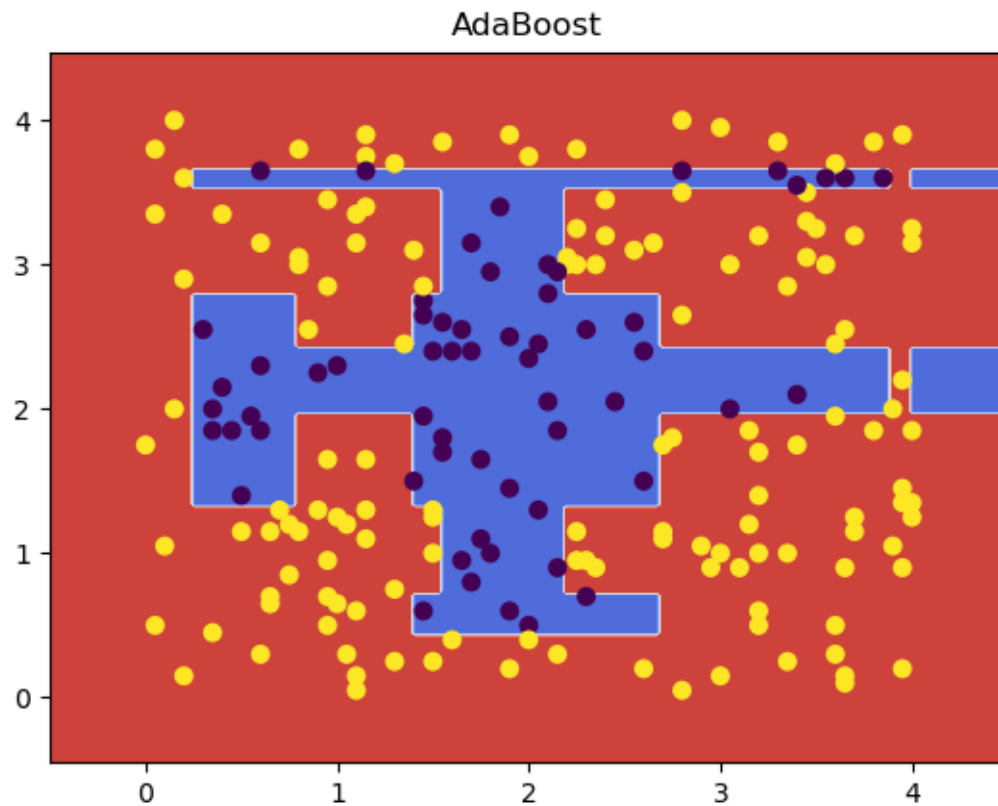
The decision tree classifier worked extremely well. It had the highest accuracy, of 99% in the data. A decision tree is a non-linear classifier, so it was able to handle this case well. Since a decision tree can split data, it can use decisions such as $A > 1.5$ as an example boundary, which works well for this board.

4. Random Forest



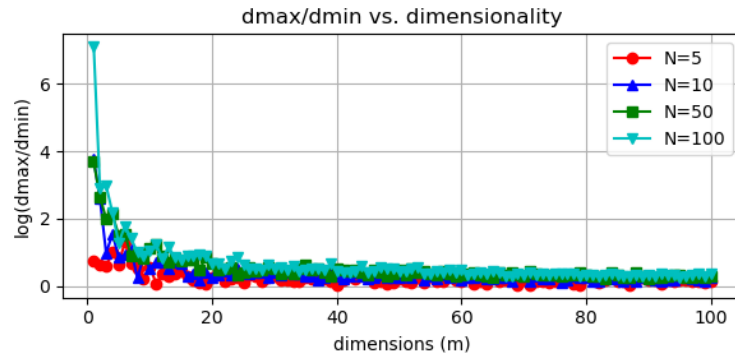
The random forest had similar results to the KNN and decision tree algorithms, having an accuracy of 94%. A random forest combines multiple decision trees and averages the results. It seemed like the decision tree was very well tuned, achieving an accuracy of 99%. Other decision trees seem unlikely to obtain better results, so averaging them is bound to result in worse results. This was a unique case where the simplicity of the structure of the data made the decision tree a better classifier.

5. Adaboost



The AdaBoost algorithm really struggled with the data. It seemed like it overfit the data, drawing borders that were extremely specific. Even during the training portion, the best score achieved was 67%. While this is somewhat better than the logistic regression, it is still not great. However, inspecting the plot, we can see that the AdaBoost algorithm seemed pretty close to getting the tic-tac-toe pattern that we expected, so with more parameter tuning, it would be possible for this to work and not overfit the data.

Part II: High Dimensionality: Curse or blessing?



As we can see in the image, $d_{\max} / d_{\min}$ is large at lower dimensions, and then quickly becomes smaller and smaller as the number of dimensions increase. This shows that when the dimensions are smaller, it is easy to have clear distinction on what data is close/far from each other. Then, as we increase the dimension, $d_{\max} / d_{\min}$ tends towards 1. This means that the Euclidian distances between all points are about the same, and we can not have a clear distinction between points.

Having high dimensionality has its own tradeoffs and pitfalls. Pedro Domingos' article, "A Few Useful Things to Know about Machine Learning", dives into these topics. Domingo states that his findings are similar to those that we saw in the figure above. "Even with a moderate dimension of 100 and a huge training set of a trillion examples, the latter covers only a fraction of about 10^{-18} of the input space" (Domingos, 2012). This gives great insight into the amount of data that is needed to be able to properly train a model with a large number of dimensions. Being unable to use data with a high number of dimensions could be extremely detrimental to technologies in fields such as medicine, where there are hundreds of individual factors that can impact a patient's diagnosis because algorithms like KNN would not be able to properly classify the data.

However, there are cases where having higher dimensions is helpful. The Wikipedia article, "Curse of dimensionality", discusses how high dimensionality can be a blessing. An example of how the high dimensionality can be useful is the "linear separability of a random point from a large finite random set" ("Curse of dimensionality"). This means that there can be a hyperplane in a data set with a large amount of dimension such that the hyperplane only touches the one data point. This shows that there is ways to separate the data if the hyperplane is created correctly. Further, in a data set with high dimensionality, "the squared distance of random points to a selected point is, with high probability, close to the average" ("Curse of dimensionality"). This is saying that in a data set with high dimensionality, if the data is truly random, the distance of it to any point will be close to random. Knowing this, if there is a series of points that are clustered far away from the average distance, there is reason to believe that it is a grouping that is not random.

High dimensionality has its own benefits and issues. Knowing how to look at the data and assess it is extremely important to truly understand it, regardless of the dimensionality. Using the wrong classifier on a set of data with high dimensionality, such as KNN, will yield in unusable results. However, if the user can make a classifier to look for unique details such as deviations from the Euclidian distance, some useful knowledge can be gained.